Практическая работа №22
Использование элементов отображения списков и табличных данных

1 Цель работы

1.1 Научиться выполнять привязку и отображение табличных данных в приложениях Avalonia UI.

2 Литература

2.1 Avalonia documentation – Текст : электронный // AvaloniaUI, 2024. – URL: https://docs.avaloniaui.net/

3 Подготовка к работе

3.1 Повторить теоретический материал (см.п.2).

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер.

5 Задание

5.1 Отображение списков

5.1.1 Разработать класс User, в котором определить свойства Login и Password

5.1.2 Добавить в приложение новую страницу для отображения списка пользователей UserListView и UserListViewModel для нее.

5.1.3 На странице расположите элемент управления ListBox, привяжите его ItemsSource к свойству ObservableCollection<User> Users (создайте это свойство во UserListViewModel и заполните тестовым набором значений)

5.2 Шаблоны элементов списка

5.2.1 Определите шаблон для элементов списка: в строке отображайте логин, пароль и кнопку «Удалить»

5.2.2 Определите привязки для элементов управления. Внутри списка для каждого его элемента контекстом привязки будет являться объект User.

5.2.3 Для привязки кнопки «Удалить» необходимо указать более сложный путь привязки через родительский элемент:

```
{Binding $parent[UserControl].DataContext.DeleteCommand}
```

Таким образом будет осуществлена привязка данных к контексту данных страницы.

5.2.4 При нажатии пользователем на кнопку запись User должна удаляться из списка

5.3 Добавление элементов в список

5.3.1 Добавить на страницу кнопку Добавить и поля ввода для указания логина и пароля.

5.3.2 Реализовать вставку нового пользователя в список. Данные нового пользователя брать из полей ввода.

5.4 Работа с DataGrid

5.4.1 Добавить в проект nuget-пакет Avalonia.Controls.DataGrid

5.4.2 В файле App.axaml в раздел Application.Styles добавить

```
<StyleInclude
    Source="avares://Avalonia.Controls.DataGrid/Themes/Fluent.xaml"/>
```

5.4.3 Добавить на страницу элемент DataGrid, привязать к нему список пользователей

5.4.4 Настроить отображение колонок с информацией о пользователе

```
<DataGrid.Columns>
     <DataGridTextColumn  Header="Логин"  Binding="{Binding  Login}"/>
     ...
</DataGrid.Columns>
```

5.4.5 Добавить колонку с кнопкой «Удалить» при помощи DataGridTemplateColumn

5.4.6 Протестировать работу свойств DataGrid

```
CanUserReorderColumns="True"
CanUserResizeColumns="True"
CanUserSortColumns="True"
```

6 Порядок выполнения работы

6.1 Выполнить все задания из п.5.

6.2 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

8 Контрольные вопросы

8.1 Какие элементы управления используются для отображения списков в Avalonia UI?

8.2 Как изменить шаблон элемента списка?

8.3 Как осуществить привязку данных к элементам списка?

Практическая работа №23
Передача и обработка данных в приложении

1 Цель работы

1.1 Научиться выполнять обработку данных в приложениях Avalonia UI.

1.2 Научиться передавать данные между частями приложения при помощи внедрения зависимостей в приложениях Avalonia UI.

2 Литература

2.1 Avalonia documentation – Текст : электронный // AvaloniaUI, 2024. – URL: https://docs.avaloniaui.net/

3 Подготовка к работе

3.1 Повторить теоретический материал (см.п.2).

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер.

5 Задание

5.1 Конфигурация приложения

5.1.1 Установите пакеты:
Microsoft.Extensions.Configuration,
Microsoft.Extensions.Configuration.Json,

5.1.2 В файле App.axaml.cs добавьте свойство

```
public static IConfiguration Configuration { get; private set; }
```

5.1.3 Добавьте в проект файл appsettings.json

```
{
  "ApiKeys": {
    "SomeApi": "YourApiKey"
  }
}
```

5.1.4 Создайте в файле App.axaml.cs ConfigurationBuilder для настроек вашего приложения

```
ConfigurationBuilder builder = new ConfigurationBuilder();
//добавляем файл с настройками
builder.AddJsonFile("appsettings.json");
Configuration = builder.Build();
```

5.1.5 При помощи обращения к конфигурации приложения отобразите данные из конфигурации на странице приложения

```
var key = App.Configuration.GetSection("ApiKeys")["SomeApi"];
```

5.2 Внедрение зависимостей

5.2.1 Установите пакет Microsoft.Extensions.DependencyInjection

5.2.2 В файле App.axaml.cs добавьте свойство

```
public static ServiceProvider Services { get; private set; }
```

5.2.1 Создайте в файле App.axaml.cs ServiceCollection для вашего приложения

```
var collection = new ServiceCollection();

//Добавить сервисы

Services = collection.BuildServiceProvider();
```

5.3 Модифицировать сервис навигации из ПР№3 с использованием внедрения зависимостей

5.3.1 Добавить в сервис навигации наследование от ObservableObject, а также вызовы OnPropertyChanged() для свойства текущей вершины стека навигации при вызове методов перехода между страницами.

5.3.2 Добавить перегруженный метод, получающий страницу для навигации через внедрение зависимостей с помощью `GetRequiredService()`

```
public void NavigateTo<T>(Action<T>? action = null) where T :
ViewModelBase
{
    var viewModel = App.Services.GetRequiredService<T>();
    NavigateTo(viewModel, action);
}
```

5.3.3 Зарегистрируйте сервис навигации и страницы в приложении в коллекции сервисов

```
//Singleton - один на все приложение
collection.AddSingleton<NavigationService>();
collection.AddSingleton<MainViewModel>();
//Transient - создается новый при каждом обращении
collection.AddTransient<RegistrationViewModel>();
collection.AddTransient<AuthorizationViewModel>();
```

5.4 Применение внедрения зависимостей

5.4.1 В конструкторы `RegistrationViewModel` и `AuthorizationViewModel` добавьте параметр `NavigationService navigation`, значение параметра присвойте приватному полю класса. Аналогичные действия выполните в MainViewModel.

5.4.2 На страницах расположите кнопки для перехода от одной страницы к другой, к ним необходимо привязать команды, которые будут вызывать метод навигации. В параметры передайте значение свойства для настройки свойств целевой страницы.

```
_navigation.NavigateTo<AuthorizationViewModel>(
    x=>x.Login = this.Login);
```

5.4.3 В MainViewModel измените структуру отображения страницы

```
public ViewModelBase CurrentPage => navigation.Current;
```

5.4.4 В конструкторе добавьте подписку на событие PropertyChanged

сервиса навигации, и соответствующий обработчик, который будет вызывать обновление значения CurrentPage

```
navigation.PropertyChanged += navigation_PropertyChanged;
```

```
private void navigation_PropertyChanged(object? sender, PropertyChangedEventArgs e)
{
    OnPropertyChanged(nameof(CurrentPage));
}
```

5.5 Конверторы значений

5.5.1 Для кастомизации отображения и преобразования привязанных данных используются конверторы значений. Создайте новый класс CelsiusToFarenhateConverter, наследующий IValueConverter

```
public class CelsiusToFarenhateConverter : IValueConverter
{
    public object? Convert(object? value, Type targetType, object? parameter, CultureInfo culture)
    {
        if (value == null)
            return null;
        return value; //преобразуйте value из градусов C в градусы F
    }

    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
    {
        return value; //преобразуйте value из градусов F в градусы C
    }
}
```

5.5.2 Для применения конвертера добавьте на страницу

```
<UserControl.Resources>
    <local:CelsiusToFarenhateConverter x:Key="CelsiusToFarenhate"/>
</UserControl.Resources>
```

Применение в привязке:

```
{Binding Celsius, Converter={StaticResource CelsiusToFarenhate}}
```

5.5.3 Проверьте работу конвертора

6 Порядок выполнения работы

6.1 Выполнить все задания из п.5.

6.2 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

8 Контрольные вопросы

8.1 Для чего необходимо внедрение зависимостей
8.2 Как использовать конфигурацию IConfiguration
8.3 Как получить сервис из коллекции сервисов.
8.4 Для чего применяются конверторы значений

Практическая работа №24
Настройка стиля интерфейса приложения

1 Цель работы

1.1 Изучить основы стилизации интерфейса Avalonia UI.

2 Литература

2.1 Avalonia documentation – Текст : электронный // AvaloniaUI, 2024. – URL: https://docs.avaloniaui.net/

3 Подготовка к работе

3.1 Повторить теоретический материал (см.п.2).

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер.

5 Задание

5.1 Применение стилей в Avalonia

5.1.1 Создайте новый проект Avalonia

5.1.2 В UserControl.Styles создайте новый стиль. Используйте селектор «Button» для задания общего стиля всем кнопкам. Измените цвет фона и шрифт. Протестируйте стиль.

5.1.3 Добавьте стиль для TextBox, который применяется только к тем, у кого установлен атрибут Name="Title" (используйте селектор «TextBox#Title»). Протестируйте стиль.

5.1.4 Создайте стиль, который применяется только к кнопкам внутри StackPanel. Измените цвет фона и шрифт. Используйте селектор «StackPanel Button». Протестируйте стиль.

5.1.5 Создайте стиль для TextBlock, который применяет курсивное начертание, если его родитель Border содержит класс highlight (.highlight TextBlock). Протестируйте стиль.

5.2 Применение псевдоклассов

Так как стили большинства псевдоклассов определены в Fluent-теме Avalonia, а элементы состоят из других элементарных элементов, для переопределения свойств, используемых элементов требуется применять селекторы шаблона, например для изменения цвета границы у disabeled TextBox необходимо использовать селектор "TextBox:disabled /template/ Border#PART_BorderElement". Необходимые свойства можно узнать, изучив стандартные стили и шаблоны элементов управления Fluent:

https://github.com/AvaloniaUI/Avalonia/tree/master/src/Avalonia.Themes.Fluent/Controls

5.2.1 Создайте стиль для кнопки, который изменяет её цвет при наведении (:pointerover /template/ ContentPresenter#PART_ContentPresenter).

5.2.2 Добавьте стиль для TextBox, который меняет цвет границы при

фокусировке (:focus /template/ Border#PART_BorderElement).

5.2.3 Создайте стиль для ListBoxItem, который меняет фон при выборе (:selected /template/ ContentPresenter#PART_ContentPresenter).

5.2.4 Создайте стиль для ListBoxItem, который определяет цвет фона для четных элементов (:nth-child(even)).

5.3 Привязка к стилям

5.3.1 Создайте 2 стиля для TextBlock, установите в них различный цвет текста.

5.3.2 Создайте класс MoneyOperation со следующими свойствами описание, сумма, является ли операция доходом или расходом.

5.3.3 Отобразите на странице список MoneyOperation, в шаблоне ItemTemplate которого определите цвет метки с суммой при помощи стиля на основании привязки к свойству логического типа. Пример привязки:

```
<Style Selector="TextBlock.class1">
   <Setter .../>
</Style>
<Style Selector="TextBlock.class2">
   <Setter .../>
</Style>
<TextBlock
             Classes.class1="{Binding IsClass1}"
          Classes.class2="{Binding !IsClass1}"
          Text="{Binding Title}"/>
```

5.4 Стилизация элементов дочерних классов

5.4.1 Создайте стиль, определяющий фон для всех элементов-наследников Button (:is(Button))

5.5 Стилизация на основе значений

При помощи специализированных селекторов можно создавать стили, применяющиеся на основании значений свойств элементов, например «Button[IsCancel=true]» или «TextBlock[(Grid.Row)=0]»

5.5.1 Создайте стиль, который меняет цвет шрифта для TextBlock в которых установлен размер шрифта 16.

5.6 Изменение внешнего вида приложения с помощью Material Avalonia

5.6.1 Создать новый проект. Установить в проект nuget-пакет Material.Avalonia

5.6.2 В файле App.axaml добавить пространство имен

```
xmlns:themes="clr-namespace:Material.Styles.Themes;assembly=Material.Styles"
```

5.6.3 В стили приложения добавить

```
<themes:MaterialTheme BaseTheme="Light" PrimaryColor="Purple" SecondaryColor="Lime"/>
```

5.6.4 В MainView добавьте StackPanel и в ней добавьте следующие элементы управления

TextBlock

TextBox

Button
CheckBox
два RadioButton
DatePicker
ComboBox с двумя вложенными ComboBoxItem

С некоторыми особенностями и дополнительными возможностями использования элементов вы можете ознакомиться по ссылке:

https://github.com/AvaloniaCommunity/Material.Avalonia/tree/master/Material.Avalonia.Demo/Pages

5.6.5 Сравнить внешний вид стандартных элементов управления и стилизованных

5.6.6 В MainView добавить

```
xmlns:a="clr-namespace:Material.Styles.Assists;assembly=Material.Styles"
```

5.6.7 Используя приведенные настройки изменить внешний вид TextBox и ComboBox на Outlined

```
<TextBox a:TextFieldAssist.Label="TextBox"
         UseFloatingWatermark="True"
         Theme="{StaticResource OutlineTextBox}"
         Classes="outline densed"
         />

<ComboBox Theme="{StaticResource MaterialOutlineComboBox}"
          a:ComboBoxAssist.Label="ComboBox">
    <ComboBoxItem Content="Option 1"/>
    <ComboBoxItem Content="Option 2"/>
</ComboBox>
```

6 Порядок выполнения работы

6.1 Выполнить все задания из п.5.

6.2 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

8 Контрольные вопросы

8.1 Какие селекторы стилей существуют в Avalonia

8.2 Как подключить в проект набор элементов Material

8.3 Для чего используются псевдоклассы селекторов